

Adora — GPU Forward Synthesis

As-built notes · Ivan Milic · 2026-09-19 · session summary

This session took the existing forward synth to the **GPU**: confirmed the `adora-develop` JAX is a CUDA build and sees the H100s, ran `synth3d.py` on-device *unchanged*, added a convenience wrapper, and benchmarked real MURaM cubes. Complements the CPU parallel/memory notes in [adora_6_parallel_memory.pdf](#). Housekeeping: all `.tex/.pdf` summaries moved into [3dtesting/docs/](#).

1. Environment & GPU readiness — verified

`adora-develop`'s **JAX 0.7.1** is the **CUDA12** build (`jax-cuda12-plugin/-pjrt` + bundled `nvidia-*-cu12` libs); `jax.default_backend()=="gpu"`. Node has **3× NVIDIA H100 80 GB** (driver 550.90.07, CUDA 12.4). **No code change** was needed: `synth3d.py` is pure `jax.jit/vmap`, so it compiles to whichever backend is default. Two env vars steer it (set *before* `import jax`):

- `CUDA_VISIBLE_DEVICES=1` — pick one physical GPU (GPU 0 was busy with another user); inside the process it rennumbers to `cuda:0`.
- `XLA_PYTHON_CLIENT_PREALLOCATE=false` — stop XLA grabbing ~75% of the card up front; polite on a shared node.

Note: `synth3d.mp.py` is the *CPU-only* path (core pinning + per-core JAX processes); for GPU you call `synth_cube` on a big batch, not the pool.

2. New wrapper `synth_cube_map` (mp-style call, GPU)

Added to `synth3d.py`: a GPU twin of `synth_cube_mp` with the *same* interface — `synth_cube_map(waves, dz, cube, linelist=..., adata=None)` where `cube` is the dict of the 8 KEYS, each `(nx,ny,nz)` or `(npix,nz)`. It builds `adata`, flattens, calls `synth_cube`, and restores `(nx,ny,4,nwave)`. Unlike the mp version it returns a **JAX array on the device** (composes with `grad/jit`; `np.asarray()` to pull to host) and takes no pool knobs. **Caveat:** it runs the *whole* cube in one call — fine for small maps, but see §3: a real cube must be column-batched (auto-batching is the recommended next step).

3. Where the memory goes on GPU (same lesson, one device)

One `synth_cube` call is the triply-nested `vmap` (`columns×wavelength×depth`) fused into a single XLA program, so the polarised opacity `chi` of shape `(cols ×  $n_\lambda$  ×  $n_z$  × 4 × 4)` is live at once:

$$\text{bytes}(\text{chi}) \approx \text{cols} \cdot n_\lambda \cdot n_z \cdot 16 \cdot 8.$$

For the `skip=8` cube (36 864 cols, $n_\lambda=201$, $n_z=121$) that is **~115 GB** — over one H100. So we **batch columns** exactly as the mp path tiles them, choosing $B = \lfloor \text{budget} / (n_\lambda n_z \cdot 128) \rfloor$; an 8 GB budget gives $B \approx 2560$ columns (compiles once on batch 1, reused warm).

4. Benchmarks (GPU #1, one idle H100, x64)

Reproduced the last mp run (`3D_full_subdomain` iter 0, `muramsub`, 201 waves @ 630.1–630.3 nm), `synth` on-device, $B=2560$:

run	columns	warm synth	incl. compile	cols/s
<code>skip=8</code> (192 ²)	36 864	3.2 s	5.6 s	11 532
<code>quarter skip=1</code> (768 ²)	589 824	49 s	52.6 s	11 958

Throughput is flat at **~11.9k cols/s** (~0.084 ms/column; a full 121-layer, 201- λ polarised Stokes synthesis each), i.e. **linear scaling** — the H100 is saturated and per-batch overhead is negligible. One-time JIT ≈ 2.4 s. **Correctness:** the `skip=8` GPU cube matches the CPU-mp reference [adora_synth/synthesized_cube_djordje.npy](#) to $\max|\Delta|=7.5\text{e-}12$, $\text{mean}=2\text{e-}14$ — bit-identical up to fp64 round-off.

Extrapolation: the full 1536² domain (~2.36M cols) is **~3.3 min** of warm synth. **Loading** the snapshot from disk is ~28 s cold (~3 s once page-cached) — at full resolution the MURaM I/O, not the RT, is the emerging bottleneck.

5. Outputs written

(all under `/dat/milic/adora_synth/`)

- [synthesized_cube_gpu.npy](#) — GPU, `skip=8`, (192, 192, 4, 201), 237 MB.
- [synthesized_cube_gpu_quarter_768x768.npy](#) — GPU, quarter full-res, (768, 768, 4, 201), 3.79 GB.
- mp reference [synthesized_cube_djordje.npy](#) left untouched. Axes for all: $(x, y, \text{Stokes } IQUV, \lambda)$ on the same 201-point grid.

6. What's next

- **Auto-batching GPU path:** make `synth_cube_map` chunk columns from an $n_\lambda n_z$ memory budget (the on-device analogue of the mp `tile` fix) so it can't OOM.
- **Multi-GPU:** shard columns across the 3 H100s (`jax.shard_map/pmap` over *GPU devices* — unlike the CPU `pmap` dead end, this is the right tool here). $\sim 3\times$ for full-domain runs.
- **I/O:** at full res the load dominates — cache/convert the cube once, or read only the needed crop.
- **Response cube on GPU:** `vmap(jacrev(...))` through the same kernel; wire the differentiable `synth_cube` into the neural-field / PINN inversion loop.